

PLAYLINK

Dokumentacja testowa

Wyniki, przypadki testowe i materiały dowodowe

Stan wydania

Backend: 135/135 testów - PASS
Frontend: 61/61 testów - PASS
Pokrycie linii backendu: 89,86% (próg CI: 80%)
Build, Ruff i ESLint: PASS

Zespół PlayLink

19 czerwca 2026

Dokument opracowano na podstawie wykonywalnych testów, konfiguracji CI/CD, wyników testu obciążeniowego i ręcznej weryfikacji funkcji realtime.

Spis treści

Spis treści	2
1. Wprowadzenie	3
2. Zakres testów	3
2.1 Rodzaje zastosowanych testów	3
3. Środowisko testowe	4
3.1 Dane i izolacja	4
4. Podsumowanie wykonania	4
4.1 Przykładowe fragmenty kodu testów	5
5. Katalog przypadków testowych	7
5.1 Testy jednostkowe i komponentowe	7
5.2 Testy integracyjne i API	8
5.3 Testy funkcjonalne i kontraktowe frontendu	10
5.4 Testy WebSocket i realtime	11
5.5 Testy bezpieczeństwa	12
5.6 Testy wydajnościowe	14
5.7 Testy migracji i procesu CI	15
6. Wyniki testów wydajnościowych	16
7. Regresje i automatyzacja	16
7.1 Regresja zamykania pokoju z Admin Preview	16
7.2 Pipeline CI/CD	16
8. Ograniczenia i rekomendacje	16
9. Podsumowanie	17
Załącznik A. Polecenia odtworzeniowe	17

1. Wprowadzenie

PlayLink jest aplikacją LFG (Looking For Group) służącą do tworzenia krótkotrwałych sesji gier, wyszukiwania uczestników i komunikacji w czasie rzeczywistym. System składa się z backendu FastAPI, frontendu SvelteKit, bazy PostgreSQL oraz kanałów WebSocket. Tożsamość użytkownika jest potwierdzana kryptograficznie za pomocą BIP39 i ECDSA.

Celem dokumentacji jest przedstawienie strategii testowania, środowiska, wyników zbiorczych oraz reprezentatywnych przypadków testowych w formie umożliwiającej ich powtórzenie i audyt. Każdy przypadek wskazuje konkretne źródło w repozytorium albo procedurę manualną.

2. Zakres testów

Zakres obejmuje następujące obszary:

- uwierzytelnianie challenge-response, JWT i zarządzanie sesją;
- tworzenie, przeglądanie, dołączanie i opuszczanie pokoi;
- czat, lista uczestników, wydarzenia i RSVP w czasie rzeczywistym;
- operacje administratora, w tym zamykanie pokoi i usuwanie uczestników;
- walidację danych, integralność podpisów i odporność na replay;
- migracje bazy, wydajność endpointu listy pokoi i automatyzację CI/CD.

Poza zakresem pozostają długotrwałe testy soak, formalny audyt penetracyjny przez zewnętrzny zespół oraz pomiary produkcyjne przy setkach rzeczywistych użytkowników. Ograniczenia te opisano ponownie w rozdziale 8.

2.1 Rodzaje zastosowanych testów

Rodzaj	Narzędzie	Zakres	Dowód
Jednostkowe	pytest, Vitest	Funkcje, walidatory, store'y	UNIT-01..03
Integracyjne / API	FastAPI TestClient	REST, baza, logika domenowa	INT-01..04
Funkcjonalne / kontraktowe	Vitest	Loadery i akcje SvelteKit	FUN-01..03
WebSocket / realtime	TestClient WS	Czat, roster, event, kick	RT-01..03
Bezpieczeństwa	pytest, eth_account	JWT, nonce, role, podpisy	SEC-01..04
Wydajnościowe	load_test.py, manualne	REST i opóźnienie realtime	PERF-01..03
Migracyjne / CI	Alembic, GitHub Actions	Schemat i bramki jakości	MIG-01, CI-01

3. Środowisko testowe

Element	Konfiguracja
Backend	Python 3.14, FastAPI 0.135.1, pytest 9.0.2, pytest-cov 7.1.0
Frontend	SvelteKit 2, Svelte 5, TypeScript 5.9, Vitest 4.1.9, jsdom 29.1.1
Baza testów jednostkowych	SQLite in-memory, osobna baza dla każdej funkcji testowej
Baza integracyjna / migracje	PostgreSQL 18 w Dockerze; Alembic upgrade/downgrade/check
Kontenery	Docker Compose: db, backend, frontend
System wykonawczy	Windows 11 + Docker Desktop (Linux containers)
CI	GitHub Actions: Ubuntu, Python 3.14, Node 22, Bun
Data pomiarów	19 czerwca 2026

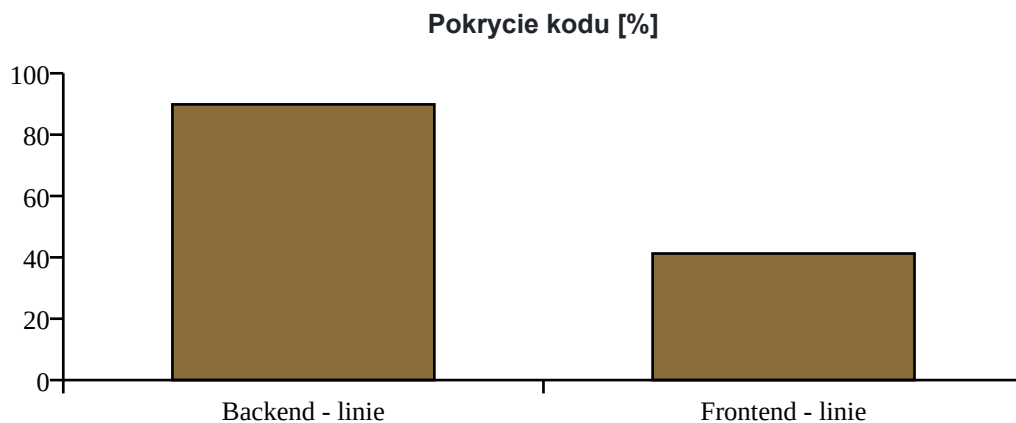
3.1 Dane i izolacja

Fixture backendu tworzy bazę SQLite w pamięci, zakłada wszystkie tabele, dodaje pięć referencyjnych gier i nadpisuje zależność `get_session`. Dzięki temu każdy test pracuje na izolowanym stanie. Rate limiting jest wyłączony w większości testów funkcjonalnych, aby nie maskował zachowania domenowego.

Test obciążeniowy wykonano na osobnym kontenerze backendu połączonym z lokalnym PostgreSQL. Limit REST podniesiono wyłącznie na czas pomiaru, aby mierzyć przepustowość endpointu, a nie celowo ustawioną ochronę 10/min.

4. Podsumowanie wykonania

Obszar	Wynik	Status
Backend pytest	135 testów; 135 zaliczonych	PASS
Backend coverage	89,86% linii; próg CI 80%	PASS
Frontend Vitest	10 plików; 61 testów; 61 zaliczonych	PASS
Frontend coverage	41,26% linii; wynik informacyjny	PASS
Ruff	check + format --check	PASS
ESLint	frontend/src	PASS
Docker build	obrazy backendu i frontendu	PASS
Test obciążeniowy	500 żądań w dwóch profilach; 0 błędów	PASS



Dowód wykonania zestawów automatycznych

pytest: 135 passed; total coverage 89.86%
Vitest: 10 test files passed; 61 tests passed
Ruff: All checks passed; 42 files already formatted
ESLint: exit code 0

4.1 Przykładowe fragmenty kodu testów

Poniższe fragmenty pochodzą bezpośrednio z repozytorium. Pokazują sposób budowania asercji, izolowania danych, testowania komunikacji WebSocket oraz odtwarzania błędu zamykania pokoju na warstwie frontendu.

Test jednostkowy funkcji hashującej nonce

Źródło: backend/tests/test_helpers.py

```
def test_hash_nonce_is_stable_and_not_plaintext():
    nonce = "challenge-value"
    hashed = main.hash_nonce(nonce)

    assert hashed == main.hash_nonce(nonce)
    assert hashed != nonce
    assert len(hashed) == 64
```

Test integracyjny kaskadowego zamknięcia pokoju

Źródło: backend/tests/test_admin.py

```
res = client.delete("/rooms/doomed", headers=headers)
assert res.status_code == 200
assert res.json()["status"] == "closed"

assert session.exec(select(Room)).first() is None
assert session.exec(select(Message)).all() == []
assert session.exec(select(RoomEvent)).all() == []
assert session.exec(select(RoomEventRsvp)).all() == []
assert session.exec(select(RoomMember)).all() == []
```

Test aktualizacji rosteru przez WebSocket

Źródło: backend/tests/test_room_events.py

```
with client.websocket_connect(
    f"/ws/rooms/lobby-1/chat?token={creator_t}"
) as ws:
    ws.receive_json() # history
    client.post(
        "/rooms/lobby-1/join",
        headers=_auth_headers("0xJoiner"),
    )

    frame = ws.receive_json()
    assert frame["type"] == "roster_update"
    addrs = [member["address"] for member in frame["members"]]
    assert "0xCreator" in addrs
    assert "0xJoiner" in addrs
```

Test regresyjny zamknięcia pokoju z nazwą zawierającą spację

Źródło: frontend/src/test/rooms.name.page.server.test.ts

```
const result = await actions.closeRoom({
  cookies: cookies(),
  params: { name: 'Lobby 2' }
} as any);

expect(fetchMock).toHaveBeenCalledWith(
  'http://localhost:8000/rooms/Lobby%202',
  {
    method: 'DELETE',
    headers: { Authorization: 'Bearer token' }
  }
);
expect(result).toEqual({ success: true, closed: true });
```

Uruchomienie testów frontendu z pomiarem pokrycia w CI

Źródło: .github/workflows/cicd.yml

```
- uses: actions/setup-node@v4
  with:
    node-version: "22"

- name: Install dependencies
  run: bun install --frozen-lockfile
  working-directory: frontend

- name: Run tests
  run: node node_modules/vitest/vitest.mjs --coverage --run
  working-directory: frontend
```

5. Katalog przypadków testowych

Poniższe przypadki wybrano tak, aby reprezentowały wszystkie warstwy systemu. Status PASS oznacza wynik uzyskany w opisanym środowisku.

5.1 Testy jednostkowe i komponentowe

UNIT-01: Hashowanie wartości nonce

Rodzaj	Jednostkowy - backend
Cel	Potwierdzić deterministyczne hashowanie nonce i brak zapisu wartości jawnej.
Źródło / narzędzie	pytest: backend/tests/test_helpers.py::test_hash_nonce_is_stable_and_not_plaintext
Dane testowe	Dwa identyczne nonce oraz nonce o innej wartości.

Scenariusz testowy

1. Obliczyć skrót pierwszego nonce dwukrotnie.
2. Obliczyć skrót innego nonce.
3. Porównać skróty i wartość jawną.

Oczekiwany wynik	Identyczne wejście daje identyczny SHA-256; inne wejście daje inny skrót; skrót nie jest równy nonce.
Wynik rzeczywisty	Wszystkie asercje spełnione w zestawie backendowym.
Status	PASS

UNIT-02: Walidacja nazwy użytkownika

Rodzaj	Jednostkowy - backend
Cel	Zweryfikować format, wielkość liter oraz filtr niedozwolonych treści.
Źródło / narzędzie	pytest: backend/tests/test_users.py
Dane testowe	Nazwy poprawne, błędny format, wielkość liter i przykłady ze stoplisty.

Scenariusz testowy

1. Załadować stoplistę.
2. Sprawdzić nazwy poprawne i niepoprawne.
3. Zweryfikować brak fałszywych trafień dla zwykłych słów.

Oczekiwany wynik	Niepoprawne lub niedozwolone nazwy są odrzucane, poprawne są akceptowane.
Wynik rzeczywisty	Testy walidacji i stoplisty zakończone powodzeniem.
Status	PASS

UNIT-03: Obliczanie odległości i wybór lobby

Rodzaj	Jednostkowy - frontend
Cel	Sprawdzić funkcję Haversine oraz wybór najbliższej lokalizacji lobby.
Źródło / narzędzie	Vitest: frontend/src/test/lobbyLocations.test.ts
Dane testowe	Te same współrzędne, Londyn-Frankfurt oraz lista lokalizacji referencyjnych.

Scenariusz testowy

1. Obliczyć odległość punktu od siebie.
2. Obliczyć odległość Londyn-Frankfurt.
3. Wybrać najbliższe lobby dla współrzędnych Londynu.

Oczekiwany wynik	0 km dla tych samych punktów; odległość w oczekiwanym zakresie; wybór eu-west.
Wynik rzeczywisty	10 testów modułu lokalizacji przeszło pomyślnie.
Status	PASS

5.2 Testy integracyjne i API

INT-01: Pełny przepływ logowania challenge-response

Rodzaj	Integracyjny / API
Cel	Zweryfikować żądanie nonce, podpis EIP-191, weryfikację i wydanie JWT.
Źródło / narzędzie	FastAPI TestClient + eth_account: backend/tests/test_auth.py
Dane testowe	Nowy adres Ethereum, nonce wygenerowany przez backend i podpis prawidłowym kluczem.

Scenariusz testowy

1. POST /auth/request-nonce.
2. Podpisać komunikat kluczem klienta.
3. POST /auth/verify.
4. GET /users/me z otrzymanym tokenem.

Oczekiwany wynik	HTTP 200, token JWT, poprawna nazwa użytkownika i dostęp do /users/me.
Wynik rzeczywisty	Backend zwrócił token, a chroniony endpoint rozpoznał użytkownika.
Status	PASS

INT-02: Cykl życia pokoju

Rodzaj	Integracyjny / API + baza danych
Cel	Sprawdzić tworzenie, automatyczne dołączenie twórcy, dołączenie drugiej osoby i opuszczenie pokoju.
Źródło / narzędzie	pytest: backend/tests/test_rooms_lifecycle.py
Dane testowe	Dwóch użytkowników, pokój z limitem miejsc i poprawna lokalizacja lobby.

Scenariusz testowy

1. Utworzyć pokój jako pierwszy użytkownik.
2. Sprawdzić obecność twórcy w members.
3. Dołączyć drugiego użytkownika.
4. Opuścić pokój i odczytać payload.

Oczekiwany wynik	Lista członków i players_active odpowiadają kolejnym operacjom.
Wynik rzeczywisty	Stan API oraz relacji RoomMember był zgodny po każdej operacji.
Status	PASS

INT-03: Planowanie wydarzenia i RSVP

Rodzaj	Integracyjny / API + baza danych
Cel	Zweryfikować utworzenie wydarzenia, deklarację obecności i czyszczenie RSVP po zmianie terminu.
Źródło / narzędzie	pytest: backend/tests/test_room_events.py
Dane testowe	Pokój z twórcą i członkiem; przyszły przedział czasu; statusy present/maybe/absent.

Scenariusz testowy

1. Zaplanować wydarzenie jako twórca.
2. Ustawić RSVP członka.
3. Zmienić termin wydarzenia.
4. Odczytać stan wydarzenia.

Oczekiwany wynik	Wydarzenie jest zapisane, RSVP działa jako upsert, a zmiana terminu czyści stare deklaracje.
Wynik rzeczywisty	Operacje REST, stan bazy i payload wydarzenia były zgodne.
Status	PASS

INT-04: Administracyjne zamknięcie pokoju

Rodzaj	Integracyjny / API + kaskada danych
Cel	Potwierdzić usunięcie pokoju, wiadomości, wydarzenia, RSVP i powiązań członków.
Źródło / narzędzie	pytest: backend/tests/test_admin.py::test_admin_delete_room_cascades
Dane testowe	Pokój zawierający członka, wiadomość, wydarzenie i RSVP.

Scenariusz testowy

1. Zalogować administratora.
2. Wysłać DELETE /rooms/{name}.
3. Odczytać wszystkie powiązane tabele.
4. Sprawdzić zachowanie konta użytkownika.

Oczekiwany wynik	Pokój i dane zależne znikają, użytkownik pozostaje, odpowiedź HTTP 200.
Wynik rzeczywisty	Kaskada aplikacyjna zakończyła się poprawnie; wszystkie asercje spełnione.
Status	PASS

5.3 Testy funkcjonalne i kontraktowe frontendu

FUN-01: Utworzenie i usunięcie sesji użytkownika

Rodzaj	Funkcjonalny / kontraktowy frontend
Cel	Sprawdzić zapis JWT w bezpiecznym cookie oraz wylogowanie.
Źródło / narzędzie	Vitest: frontend/src/test/auth.page.server.test.ts
Dane testowe	Poprawny token JWT oraz zamockowany obiekt cookies.

Scenariusz testowy

1. Wywołać akcję login z tokenem.
2. Sprawdzić parametry cookie.
3. Wywołać logout.
4. Sprawdzić usunięcie cookie.

Oczekiwany wynik	Token zostaje zapisany jako httpOnly/sameSite, a wylogowanie usuwa sesję.
Wynik rzeczywisty	Akcje logowania i wylogowania przeszły wszystkie asercje.
Status	PASS

FUN-02: Zmiana nazwy użytkownika

Rodzaj	Funkcjonalny / kontraktowy frontend
Cel	Zweryfikować walidację formularza oraz przekazanie autoryzowanej operacji PATCH do backendu.
Źródło / narzędzie	Vitest: frontend/src/test/profile.page.server.test.ts
Dane testowe	Aktywna sesja, poprawna nazwa oraz nazwa pusta.

Scenariusz testowy

1. Załadować profil z sesją.
2. Wysłać pustą nazwę.
3. Wysłać poprawną nazwę.
4. Sprawdzić metodę, nagłówki i wynik.

Oczekiwany wynik	Pusta nazwa jest odrzucana, a poprawna przekazywana do PATCH /users/me.
Wynik rzeczywisty	Loader i akcja aktualizacji zakończyły się powodzeniem.
Status	PASS

FUN-03: Zamknięcie pokoju z Admin Preview

Rodzaj	Funkcjonalny / regresyjny frontend
Cel	Potwierdzić poprawne wywołanie nazwanej akcji SvelteKit i przekazanie DELETE do backendu.
Źródło / narzędzie	Vitest: frontend/src/test/rooms.name.page.server.test.ts; +page.svelte
Dane testowe	Pokój o nazwie 'Lobby 2', aktywna sesja administratora.

Scenariusz testowy

1. Wysłać POST do `~/closeRoom` jako FormData.
2. Zakodować nazwę pokoju w URL.
3. Przekazać Bearer JWT do backendu.
4. Zweryfikować odpowiedź `success/closed`.

Oczekiwany wynik	Brak błędu HTTP 415; backend otrzymuje DELETE <code>/rooms/Lobby%202</code> .
Wynik rzeczywisty	Kontrakt akcji przeszedł; formularz klienta ma poprawny typ danych.
Status	PASS

5.4 Testy WebSocket i realtime

RT-01: Rozgłaszanie wiadomości czatu

Rodzaj	Integracyjny WebSocket
Cel	Sprawdzić przesłanie wiadomości między dwoma członkami pokoju.
Źródło / narzędzie	pytest: <code>backend/tests/test_chat.py::test_chat_broadcast_between_members</code>
Dane testowe	Dwóch członków i dwa połączenia <code>/ws/rooms/{name}/chat</code> .

Scenariusz testowy

1. Połączyć oba klienty z ważnym JWT.
2. Odebrać historię.
3. Wysłać wiadomość z pierwszego klienta.
4. Odczytać ramkę u obu klientów.

Oczekiwany wynik	Oba połączenia otrzymują zgodną ramkę message.
Wynik rzeczywisty	Wiadomość została zapisana i rozgłoszona do obu klientów.
Status	PASS

RT-02: Synchronizacja wydarzenia i listy członków

Rodzaj	Integracyjny WebSocket
Cel	Potwierdzić ramki <code>event_update</code> , <code>rsvp_update</code> i <code>roster_update</code> po zmianach REST.
Źródło / narzędzie	pytest: <code>backend/tests/test_room_events.py</code>
Dane testowe	Aktywne połączenie czatu, planowanie wydarzenia, RSVP oraz join/leave.

Scenariusz testowy

1. Połączyć klienta z pokojem.
2. Zmienić wydarzenie lub RSVP przez REST.
3. Dołączyć albo odłączyć członka.
4. Porównać ramki z payloadem REST.

Oczekiwany wynik	Klient otrzymuje właściwy typ ramki oraz aktualny, zgodny payload.
Wynik rzeczywisty	Testy zgodności REST/WS i aktualizacji rosteru zakończone powodzeniem.
Status	PASS

RT-03: Odłączenie wyrzuconego uczestnika

Rodzaj	Integracyjny WebSocket / administracja
Cel	Sprawdzić natychmiastowe powiadomienie i zamknięcie wszystkich połączeń wyrzuconego użytkownika.
Źródło / narzędzie	pytest: backend/tests/test_kick.py::test_kick_notifies_and_disconnects_active_socket
Dane testowe	Dwa połączenia WebSocket tego samego uczestnika.

Scenariusz testowy

1. Otworzyć dwa połączenia użytkownika.
2. Wykonać akcję kick jako administrator.
3. Odebrać message, roster_update i member_kicked.
4. Sprawdzić kod zamknięcia.

Oczekiwany wynik	Oba połączenia otrzymują powiadomienia i kończą się kodem 4409.
Wynik rzeczywisty	Wszystkie ramki dotarły, a oba połączenia zostały zamknięte.
Status	PASS

5.5 Testy bezpieczeństwa

SEC-01: Odrzucenie ponownego użycia nonce

Rodzaj	Bezpieczeństwa - replay attack
Cel	Uniemożliwić ponowną autoryzację tym samym nonce i podpisem.
Źródło / narzędzie	pytest: backend/tests/test_auth_edges.py::test_verify_signature_rejects_replayed_nonce
Dane testowe	Poprawny adres, nonce i podpis wykorzystane dwa razy.

Scenariusz testowy

1. Zweryfikować podpis po raz pierwszy.
2. Powtórzyć identyczne żądanie.
3. Sprawdzić used_at w bazie.

Oczekiwany wynik	Pierwsza próba kończy się sukcesem, druga zostaje odrzucona.
Wynik rzeczywisty	Mechanizm jednorazowego nonce zablokował replay.
Status	PASS

SEC-02: Odrzucenie wygasłego JWT

Rodzaj	Bezpieczeństwa - sesja
Cel	Sprawdzić, czy wygasły token nie zapewnia dostępu do chronionego zasobu.
Źródło / narzędzie	pytest: backend/tests/test_auth_edges.py::test_get_me_rejects_expired_token
Dane testowe	Token JWT z czasem exp w przeszłości.

Scenariusz testowy

1. Utworzyć token dla istniejącego użytkownika.
2. Ustawić wygasły exp.
3. Wywołać GET /users/me.

Oczekiwany wynik	Żądanie zostaje odrzucone kodem 401.
Wynik rzeczywisty	Backend odrzucił wygasłą sesję.
Status	PASS

SEC-03: Brak eskalacji uprawnień administratora

Rodzaj	Bezpieczeństwa - autoryzacja
Cel	Potwierdzić, że sfałszowane pole is_admin w JWT nie zastępuje konfiguracji ADMIN_ADDRESSES.
Źródło / narzędzie	pytest: backend/tests/test_auth_edges.py::test_forged_admin_claim_does_not_grant_admin_access
Dane testowe	Zwykły użytkownik i token z roszczeniem is_admin=true.

Scenariusz testowy

1. Utworzyć token z fałszywym roszczeniem.
2. Wywołać administracyjne DELETE /rooms/{name}.
3. Sprawdzić stan zasobu.

Oczekiwany wynik	HTTP 403, brak wykonania operacji administracyjnej.
Wynik rzeczywisty	Backend zweryfikował adres względem własnej konfiguracji i odrzucił żądanie.
Status	PASS

SEC-04: Odrzucenie zmodyfikowanej podpisanej wiadomości

Rodzaj	Bezpieczeństwa - integralność EIP-191
Cel	Sprawdzić, czy zmiana treści po podpisaniu uniemożliwia zapis i rozgłoszenie wiadomości.
Źródło / narzędzie	pytest: backend/tests/test_chat.py::test_chat_rejects_tampered_content
Dane testowe	Podpis poprawnej treści i inna treść przesłana w ramce.

Scenariusz testowy

1. Zbudować canonical payload.
2. Podpisać treść kluczem użytkownika.
3. Zmienić content bez zmiany podpisu.
4. Wysłać ramkę WebSocket.

Oczekiwany wynik	Wiadomość nie zostaje zaakceptowana ani zapisana jako zweryfikowana.
Wynik rzeczywisty	Backend wykrył niezgodność podpisu i odrzucił zmodyfikowaną treść.
Status	PASS

5.6 Testy wydajnościowe

PERF-01: Lista pokoi - profil podstawowy

Rodzaj	Obciążeniowy HTTP
Cel	Zmierzyć stabilność i czasy odpowiedzi publicznego endpointu listy pokoi.
Źródło / narzędzie	scripts/load_test.py; lokalny Docker; limit REST podniesiony wyłącznie dla testu
Dane testowe	GET /rooms; 200 żądań; współbieżność 10.

Scenariusz testowy

1. Uruchomić backend i PostgreSQL w Dockerze.
2. Wysłać 200 żądań w 10 wątkach.
3. Zebrać kody i percentyle.

Oczekiwany wynik	Brak błędów HTTP; stabilny czas odpowiedzi poniżej 1 s dla p95.
Wynik rzeczywisty	200/200 odpowiedzi HTTP 200; średnia 53,021 ms; mediana 43,108 ms; p95 71,431 ms; 169,47 żąd./s.
Status	PASS

PERF-02: Lista pokoi - profil podwyższonego obciążenia

Rodzaj	Obciążeniowy HTTP
Cel	Sprawdzić zachowanie endpointu przy trzykrotnie większej współbieżności.
Źródło / narzędzie	scripts/load_test.py; lokalny Docker; limit REST podniesiony wyłącznie dla testu
Dane testowe	GET /rooms; 300 żądań; współbieżność 30.

Scenariusz testowy

1. Uruchomić identyczne środowisko.
2. Wysłać 300 żądań w 30 wątkach.
3. Zebrać kody i percentyle.

Oczekiwany wynik	Brak błędów serwera; wszystkie żądania obsłużone.
Wynik rzeczywisty	300/300 odpowiedzi HTTP 200; średnia 191,964 ms; mediana 124,447 ms; p95 999,600 ms; 134,07 żąd./s.
Status	PASS

PERF-03: Opóźnienie aktualizacji realtime

Rodzaj	Manualny wydajnościowy
Cel	Zweryfikować wymaganie aktualizacji interfejsu poniżej 1000 ms.
Źródło / narzędzie	Ręczny pomiar klient-klient w sieci lokalnej i uczelnianej eduroam
Dane testowe	Zmiana stanu pokoju obserwowana równocześnie w dwóch klientach.

Scenariusz testowy

1. Otworzyć dwóch klientów w jednym pokoju.
2. Wykonać zmianę generującą broadcast.
3. Zmierzyć czas do aktualizacji drugiego klienta.
4. Powtórzyć w obciążonej sieci eduroam.

Oczekiwany wynik	Aktualizacja widoczna poniżej 1000 ms.
Wynik rzeczywisty	Około 60 ms w typowych warunkach i około 600 ms w obciążonej sieci eduroam; wymaganie spełnione.
Status	PASS

5.7 Testy migracji i procesu CI

MIG-01: Budowa schematu z pełnego łańcucha Alembic

Rodzaj	Migracyjny smoke test
Cel	Potwierdzić, że czysta baza może zostać doprowadzona do bieżącej wersji.
Źródło / narzędzie	pytest: backend/tests/test_migrations.py; Alembic
Dane testowe	Pusty plik SQLite i wszystkie migracje z backend/alembic/versions.

Scenariusz testowy

1. Ustawić DATABASE_URL na bazę tymczasową.
2. Uruchomić alembic upgrade head.
3. Odczytać tabele i kluczowe kolumny.

Oczekiwany wynik	Migracja kończy się bez błędów; obecne są wszystkie wymagane tabele i kolumny.
Wynik rzeczywisty	Test migracyjny przeszedł w zestawie 135 testów backendu.
Status	PASS

CI-01: Bramka jakości przed wdrożeniem

Rodzaj	Automatyzacja CI/CD
Cel	Zweryfikować testy, lint, migracje i budowę obrazów przed deployem.
Źródło / narzędzie	.github/workflows/cicd.yml; GitHub Actions
Dane testowe	Zmiana w pull requeście lub push do main.

Scenariusz testowy

1. Uruchomić backend-lint i backend-test.
2. Uruchomić migracje na PostgreSQL.
3. Uruchomić frontend-lint i frontend-test pod Node.
4. Zbudować obrazy Docker.

Oczekiwany wynik	Każdy job kończy się sukcesem; deploy zależy od wszystkich jobów jakościowych.
Wynik rzeczywisty	Konfiguracja obejmuje wszystkie wymienione joby i jawnie włącza frontend-test do needs deploy.
Status	PASS

6. Wyniki testów wydajnościowych

Oba profile uruchomiono tym samym skryptem. Każde żądanie pobierało pełną odpowiedź GET /rooms. Nie odnotowano błędów HTTP ani timeoutów.

Metryka	Profil 200 / 10	Profil 300 / 30
Poprawne odpowiedzi	200	300
Błędy	0	0
Łączny czas	1,180 s	2,238 s
Przepustowość	169,47 żąd./s	134,07 żąd./s
Średnia	53,021 ms	191,964 ms
Mediana	43,108 ms	124,447 ms
p95	71,431 ms	999,600 ms
Maksimum	330,693 ms	1125,387 ms

Wniosek: endpoint zachowuje pełną poprawność odpowiedzi w obu profilach. Przy współbieżności 10 p95 pozostaje wyraźnie poniżej 100 ms. Przy współbieżności 30 pojawia się kolejka i p95 zbliża się do 1 s, jednak bez błędów serwera. Wynik nie jest deklaracją maksymalnej przepustowości produkcyjnej - jest powtarzalnym benchmarkiem lokalnego środowiska Docker.

Polecenie: `python scripts/load_test.py http://localhost:18000/rooms --requests 200 --concurrency 10`

7. Regresje i automatyzacja

7.1 Regresja zamykania pokoju z Admin Preview

Podczas testów manualnych wykryto odpowiedź HTTP 415 przy zamykaniu pustego pokoju. Przyczyną był pusty POST do nazwanej akcji SvelteKit bez typu formularza. Klient został zmieniony tak, aby wysyłać FormData, a kontrakt akcji closeRoom objęto testem Vitest.

Ślad regresji

Przed poprawką: POST /rooms/Lobby%202?/closeRoom -> 415 Unsupported Media Type
Po poprawce: żądanie zawiera FormData; akcja przekazuje DELETE do backendu
Test: rooms.name.page.server.test.ts - PASS

7.2 Pipeline CI/CD

Pipeline uruchamia lint i testy backendu, test migracji na PostgreSQL, lint i testy frontendu z coverage pod Node 22 oraz budowę obrazów Docker. Job deploy ma zależność od wszystkich sześciu jobów jakościowych, w tym frontend-test.

8. Ograniczenia i rekomendacje

- Brak pełnych testów E2E w prawdziwej przeglądarce; obecne testy frontendu używają jsdom i mocków.
- Frontend coverage jest raportowany informacyjnie; projekt nie ma jeszcze progu blokującego.
- Benchmark HTTP wykonano lokalnie i nie zastępuje długotrwałego testu soak ani pomiaru produkcyjnego.
- Pomiar realtime na eduroam był manualny; warto w przyszłości automatycznie logować czas wysłania i odbioru ramek.

- Testy bezpieczeństwa są testami aplikacyjnymi, nie formalnym audytem penetracyjnym.

9. Podsumowanie

System został zweryfikowany za pomocą testów jednostkowych, integracyjnych, funkcjonalnych, WebSocket, bezpieczeństwa, wydajnościowych i migracyjnych. Wszystkie opisane przypadki zakończyły się wynikiem pozytywnym. Backend przekracza wymagany próg 80% pokrycia linii, frontendowy zestaw 61 testów przechodzi w całości, a oba profile obciążeniowe zakończyły się bez błędów.

Załącznik A. Polecenia odtworzeniowe

```
Backend:
cd backend
uv run pytest --cov=main --cov=models --cov=database --cov=usernames --cov-fail-under=80
```

```
Frontend:
cd frontend
bun install --frozen-lockfile
node node_modules/vitest/vitest.mjs --coverage --run
```

```
Jakość i build:
uv run ruff check .
bun run lint
docker compose build
```